



Automating Container Builds with GitHub Actions: From Code to Docker Hub

Yucheng Zhang
yucheng.zhang@tufts.edu
Senior Bioinformatics Engineer
TTS Research Technology



Good afternoon. My name is Yucheng Zhang, and I am a Senior Bioinformatics Engineer with TTS Research Technology. Today I will introduce how we can use GitHub Actions to automate Docker container builds and publish container images to Docker Hub. The goal is to show a practical workflow that can reduce manual work, improve reproducibility, and make container maintenance easier for research computing environments.

Agenda

- ❖ Containers
- ❖ CI/CD & GitHub Actions
- ❖ GitHub Actions for Docker builds
- ❖ Demo



TTS Research Technology



Here is the plan for today. I will start with a short overview of containers and why they matter. Then I will introduce CI/CD and GitHub Actions. After that, we will look at how GitHub Actions can build and publish Docker images automatically. At the end, I will walk through a demo repository so you can see the workflow in action.

01 Containers

The first section is about containers. Before we talk about GitHub Actions, it is important to understand what we are trying to automate. In our case, the artifact we are building is a container image, so we need to know what a container is and why it is useful.

What Are Containers?

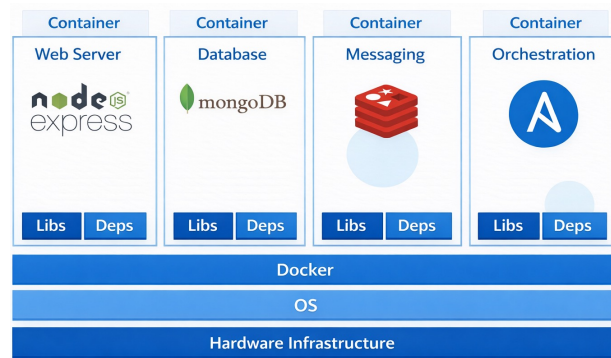
Containers are lightweight, portable, and isolated units that package an application and its **dependencies**, enabling it to run consistently across different environments.



A container packages an application together with the software environment it needs to run. That includes dependencies, libraries, runtime tools, and configuration. The main idea is portability. Instead of saying, “This works only on my laptop,” we want the same software environment to run consistently on a laptop, a server, the cloud, or an HPC system.

What can container do?

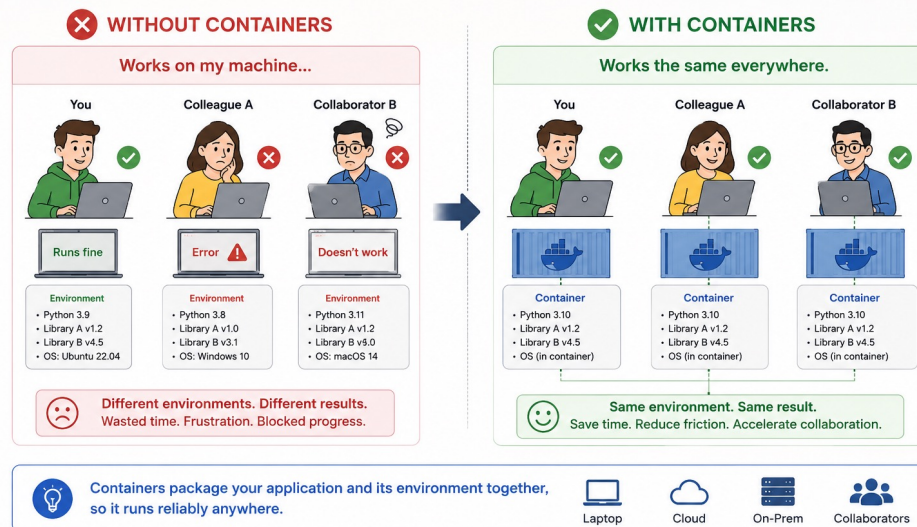
- Containerize applications
- Run each service/application with its own dependencies in separate containers



Containers allow us to package applications separately from the host operating system. Each application can have its own dependencies without interfering with other applications. For example, one container can run a web server, another can run a database, and another can run a workflow tool. This isolation makes software easier to deploy, test, and maintain.

Why Use Containers?

Because “It Works on My Machine” shouldn’t be a roadblock.



Tufts
UNIVERSITY

One of the most common problems in research computing is “it works on my machine.” Different users may have different operating systems, Python versions, library versions, or system packages. Containers help solve this by packaging the application and its environment together. This improves reproducibility and reduces the amount of time spent debugging environment problems.

What do containers mean for Tufts HPC?

Containers are a core part of the Tufts HPC ecosystem. They support a large share of the software and workflows used across research, teaching, and AI computing.

- Singularity, the container runtime, is the most heavily used application on the HPC system.
- All AI tools running on the HPC system are delivered through containers.
- Most applications that support NVIDIA GPUs are container-based.
- Bioinformatics software stacks are built on containers.
- Course computing environments that rely on HPC are provided through containers.
- R, RStudio, Jupyter Notebook, and many other commonly used applications run in containers.
- And much more.



TTS Research Technology



For Tufts HPC, containers are not just a convenience. They are a core part of how we deliver software. Many AI tools, GPU applications, bioinformatics software stacks, course environments, RStudio, Jupyter Notebook, and other applications are delivered through containers. This means container management is directly connected to user productivity on the cluster. If we can automate container builds, we can make software delivery faster and more reliable.

What Is Docker and Docker Hub?

Docker is a platform that enables developers to automate the deployment of applications inside containers. Docker Hub is a cloud-based repository where developers can find, share, and store container images.

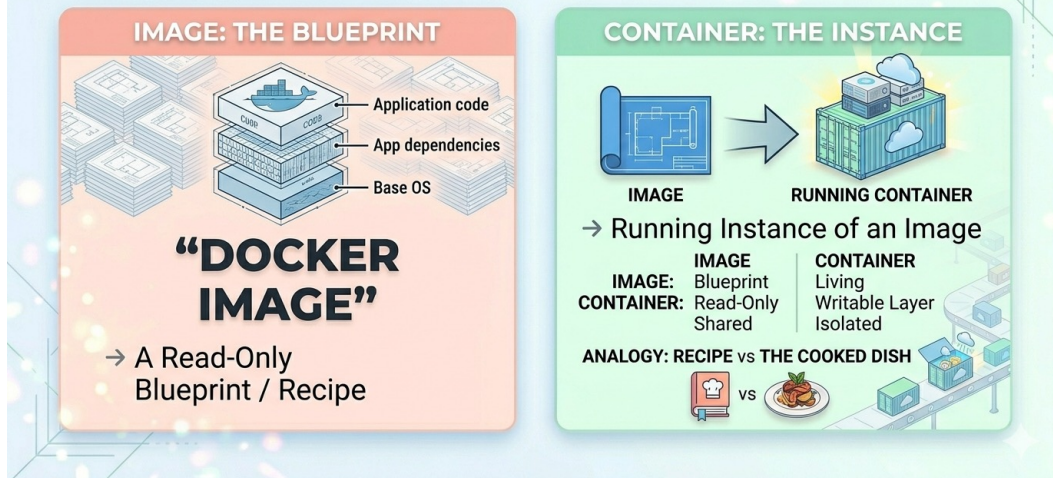


Tufts
UNIVERSITY

TTS Research Technology

Docker is one of the most widely used platforms for building and managing containers. Docker Hub is a public container registry where images can be stored, shared, and downloaded. In our workflow, GitHub Actions builds the Docker image, and then pushes the finished image to Docker Hub so that it can be reused by users, admins, or downstream systems.

DOCKER CORE CONCEPTS: IMAGE VS CONTAINER



It is important to distinguish between an image and a container. A Docker image is like a blueprint or recipe. It contains the application, dependencies, and filesystem layers. A container is a running instance of that image. So we build an image once, publish it, and then users can run containers from that image many times.

02 GitHub Actions

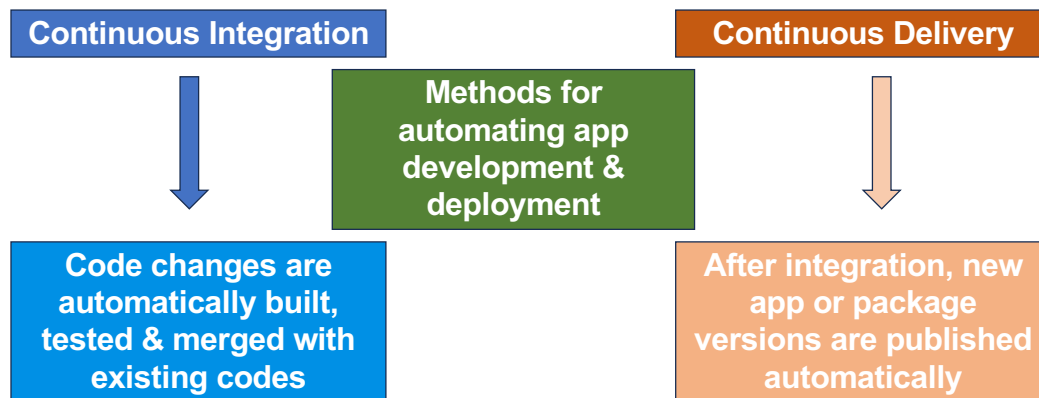


TTS Research Technology



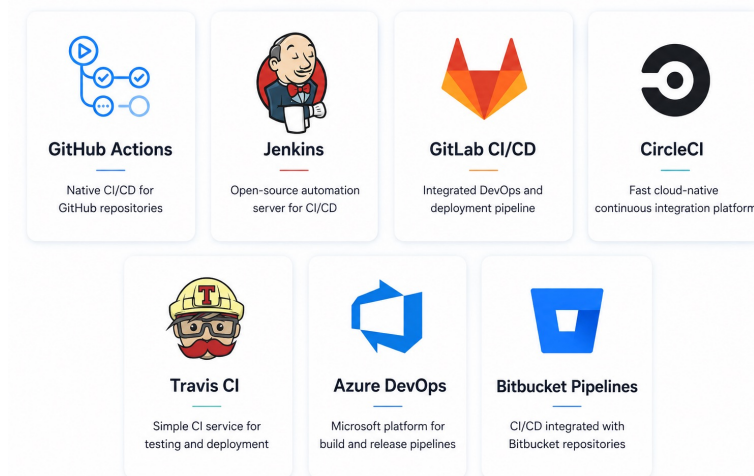
Now that we have covered containers, the next question is: how do we automate the process of building them? This is where GitHub Actions comes in. GitHub Actions lets us define automated workflows directly inside a GitHub repository.

What is CI/CD?



CI/CD stands for Continuous Integration and Continuous Delivery or Deployment. Continuous Integration means code changes are automatically built and tested. Continuous Delivery means that after the build and test steps succeed, a new version of the application or package can be published automatically. For containers, this means that when we update the Dockerfile or source code, a workflow can automatically build and publish a new container image.

Top CI/CD platforms



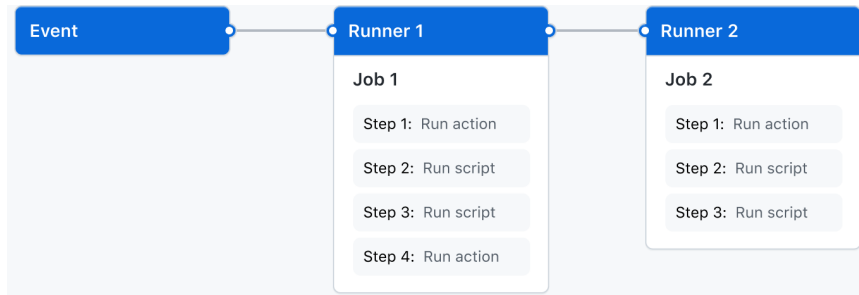
There are many CI/CD platforms, including GitHub Actions, Jenkins, GitLab CI/CD, CircleCI, Travis CI, Azure DevOps, and Bitbucket Pipelines. Today we focus on GitHub Actions because many of our container definitions and code repositories are already hosted on GitHub, so the automation can live directly next to the code.

What is GitHub Actions



GitHub Actions is GitHub's built-in automation platform. A workflow starts when something happens in the repository, such as a push, pull request, release, schedule, or manual trigger. The workflow runs jobs, and each job contains steps. Those steps can run shell commands or reusable actions from the GitHub Actions marketplace.

Components of GitHub Actions



GitHub Actions workflows run automatically when events happen in your repository. A workflow contains jobs that run on virtual machines or containers, with steps that execute scripts or reusable actions.



TTS Research Technology



This diagram shows the basic components. An event triggers a workflow. The workflow contains one or more jobs. Jobs run on runners, which are machines provided by GitHub or managed by us. Each job is made up of steps. For a Docker build workflow, these steps usually include checking out the code, logging in to Docker Hub, building the image, and pushing the image.

Components of GitHub Actions

Workflows

A **workflow** is a configurable automated process that will run one or more jobs.

Workflows are defined by a **YAML** file checked in to your repository and will run when triggered by an event in your repository, or they can be triggered manually, or at a defined schedule.

Workflows are defined in the `.github/workflows` directory in a repository. A repository can have multiple workflows



TTS Research Technology



A workflow is defined in a YAML file inside the `.github/workflows` directory of a repository. A repository can have multiple workflows. For example, one workflow might run tests, another might build documentation, and another might build and publish a Docker image. The workflow file defines when the workflow runs and what jobs and steps it should execute.

Components of GitHub Actions

Events

An **event** is a specific activity in a repository that triggers a **workflow** run.

You can also trigger a workflow to run on a schedule, by posting to a REST API, or manually.

Examples

- push: fires on every push; filter by branches, tags, or paths
- pull_request :fires on PR activity (opened, synchronize, closed)
- schedule: cron-based (e.g. nightly rebuilds to pick up base image patches)
- release: fires when a GitHub Release is published.



TTS Research Technology



Events are what trigger a workflow. A push event can run a workflow every time code is pushed. A pull request event can run tests before code is merged. A schedule event can run a workflow nightly or weekly. A release event can run when a GitHub Release is published. For production container builds, release-based triggers are often useful because they tie the container version to a formal software release.

Components of GitHub Actions

Runners

GitHub-hosted runners: free Ubuntu, Windows, and macOS VMs, but its usage has limits.

Self-hosted runners: install the actions-runner agent on any machine; useful for GPU builds, large datasets.

When to use self-hosted: image builds exceed 6 hours, require NVIDIA GPUs.

Labels: target a self-hosted runner with runs-on: [self-hosted, gpu] in your workflow



TTS Research Technology



A runner is the machine where the workflow runs. GitHub-hosted runners are temporary virtual machines provided by GitHub. They are convenient and work well for many builds. Self-hosted runners are machines we manage ourselves. They are useful when builds require special hardware, large storage, GPUs, or access to internal infrastructure. For standard Docker builds, GitHub-hosted runners are usually enough. For HPC-specific testing or deployment, self-hosted runners can be very useful.

Standard GitHub-hosted runners for public repositories

Virtual machine / container	Processor (CPU)	Memory (RAM)	Storage (SSD)	Architecture	Workflow label
Linux	1	5 GB	14 GB	x64	ubuntu-slim
Linux	4	16 GB	14 GB	x64	ubuntu-latest , ubuntu-24.04 , ubuntu-22.04
Windows	4	16 GB	14 GB	x64	windows-latest , windows-2025 , windows-2025-vs2026 , windows-2022
Linux	4	16 GB	14 GB	arm64	ubuntu-24.04-arm , ubuntu-22.04-arm
Windows	4	16 GB	14 GB	arm64	windows-11-arm
macOS	4	14 GB	14 GB	Intel	macos-16-intel , macos-26-intel
macOS	3 (M1)	7 GB	14 GB	arm64	macos-latest , macos-14 , macos-15 , macos-26



TTS Research Technology

<https://docs.github.com/en/actions/reference/runners/github-hosted-runners#standard-github-hosted-runners-for-public-repositories>

This slide shows the standard GitHub-hosted runner options. The important point is that these runners have limited CPU, memory, disk space, and runtime. For small or medium Docker images, this is usually fine. But for very large scientific containers, especially containers with large software stacks, we need to pay attention to build time, disk usage, and cache strategy.

03 GitHub actions & Docker builds



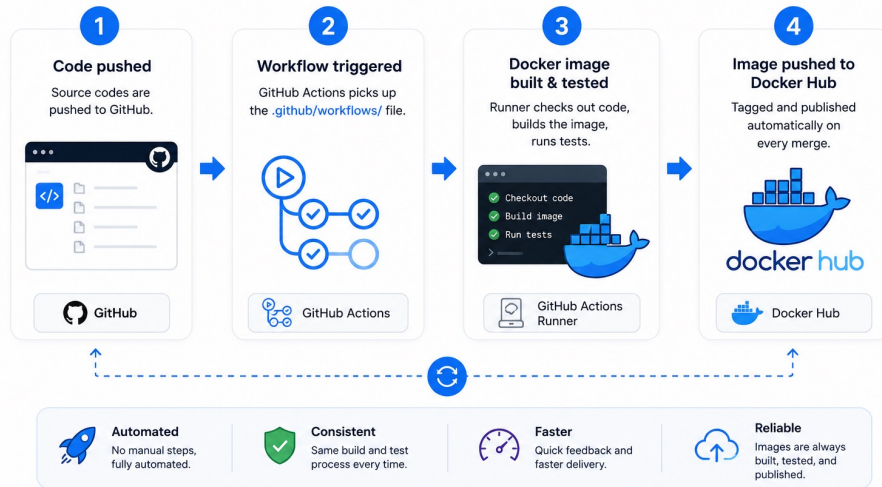
TTS Research Technology



Now we move to the main technical part: using GitHub Actions to build Docker images. The goal is to replace a manual process with an automated workflow. Instead of building and pushing images from an admin's local machine, the repository itself can build and publish the image when the right event happens.

GitHub Actions for Docker Containers

GitHub Actions automates every stage of your container lifecycle, from code commit to a published, versioned image on Docker Hub.



This is the high-level workflow. First, code is pushed or a release is published. Second, GitHub Actions detects the event and starts the workflow. Third, the Docker image is built and tested on the runner. Fourth, the image is tagged and pushed to Docker Hub. The result is a published, versioned container image that can be pulled and used elsewhere.

Anatomy of a Workflow File

Copy this into `.github/workflows/docker.yml` to get an automated, production-ready build pipeline.

▶ Trigger

Fires when a GitHub Release is published.

▶ Runner

GitHub-hosted Ubuntu VM.

▶ Secrets & Variables

Referenced with `${{ vars.NAME }}` and `${{ secrets.NAME }}`. Never printed in logs.

▶ Steps

Run in order. A failing step stops the job.

- **Checkout:** pulls your repo code into the runner
- **Login:** authenticates to Docker Hub using your stored credentials
- **Extract Metadata:** derives the image tag from the release version and attaches standard OCI labels
- **Build & Publish:** builds the image from `./Dockerfile` and pushes to `tuftstsr/rserver` with the extracted tag



TTS Research Technology

```
name: Build & Publish Docker Image
on:
  release:
    types: [published]

jobs:
  push-to-dockerhub:
    runs-on: ubuntu-latest
    permissions:
      packages: write
      contents: read
    steps:
      - uses: actions/checkout@v4
      - name: Login to Dockerhub
        uses: docker/login-action@v3
        with:
          username: ${{ vars.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_PASSWORD }}
      - name: Extract Metadata
        id: metadata
        uses: docker/metadata-action@v4
        with:
          images: |
            tuftstsr/rserver
            tags: type=semver,pattern={{version}}
      - name: Build & Publish Docker Image
        uses: docker/build-push-action@v5
        with:
          context: .
          file: ./Dockerfile
          push: true
          tags: ${{ steps.metadata.outputs.tags }}
          labels: ${{ steps.metadata.outputs.labels }}
```

Here is a real example of a workflow file. This workflow is triggered when a GitHub Release is published. It runs on an Ubuntu GitHub-hosted runner. It checks out the repository, logs in to Docker Hub, extracts metadata from the release version, and builds and publishes the Docker image. Notice that credentials are not written directly in the YAML file. They are referenced through GitHub variables and secrets.

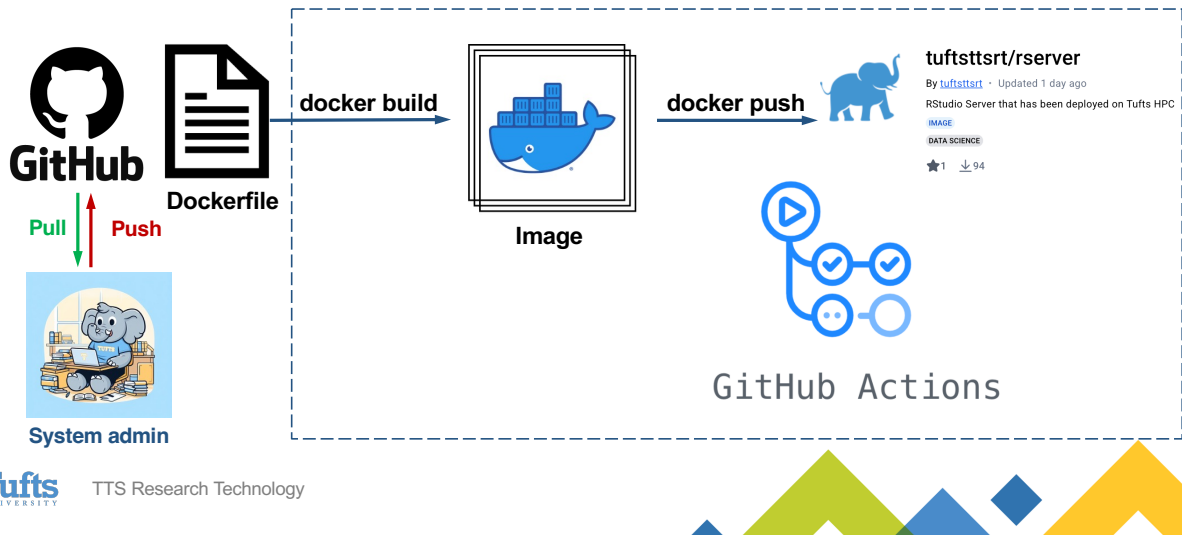


This diagram summarizes the same workflow visually. A release is published in GitHub. That release triggers the GitHub Actions workflow. The workflow checks out the code, logs in to Docker Hub, generates the image tag, builds the image, and pushes it to Docker Hub. The final result is a published image with a version tag, such as v1.2.3.

How TTS Research Technology use it?

Before: admin manually builds and pushes images

After: GitHub Actions builds, tags, and publishes images automatically



This slide shows how TTS Research Technology uses this workflow. Previously, a system administrator might manually build an image from a Dockerfile and then manually push it to Docker Hub. With GitHub Actions, the repository can handle that process automatically. The admin updates the Dockerfile or publishes a release, and GitHub Actions performs the build and push steps in a repeatable way. This reduces manual work and makes the build process easier to audit.

What this means for tufts HPC?

- Faster and more repeatable container updates
- Less manual build and push work for admins
- Dockerfile in GitHub documents exactly how each image is built
- Versioned images support reproducible research
- GitHub history gives traceability: who changed what, when, and why
- Foundation for future Apptainer/Singularity deployment



TTS Research Technology



For HPC, this workflow is not just about convenience. It also improves documentation and transparency. When the Dockerfile is stored in GitHub, we have a clear record of how the container was built: which base image was used, which packages were installed, and what commands were run. The Git history also shows who changed the container definition, when it changed, and why. That makes the container easier to maintain, easier to reproduce, and easier to troubleshoot later.

Setting Up Secrets & Credentials

Secrets keep sensitive credentials out of your code and safely stored in GitHub — never hardcode passwords or tokens in your workflow files.

- **Why secrets matter:** never hardcode credentials — leaked tokens can compromise your Docker Hub account and any connected infrastructure
- **Adding Docker Hub credentials to GitHub Secrets:** go to Settings → Secrets and variables → Actions, then add `DOCKERHUB_USERNAME` and `DOCKERHUB_PASSWORD`



TTS Research Technology



Credentials are one of the most important parts of this workflow. We should never hardcode Docker Hub usernames, passwords, or tokens directly in a workflow file. Instead, we store them in GitHub Secrets or variables. The workflow can use those values at runtime, but they are not exposed in the repository. For Docker Hub, it is better to use an access token instead of a normal account password.

What Is Layer Caching?

Each Docker instruction creates a layer. Caching saves those layers so repeated builds only redo what actually changed.

⚠ The Problem

Every GitHub Actions run starts on a brand-new virtual machine — there is no leftover cache from the previous run.

Without caching, every build reinstalls all dependencies and rebuilds all layers from scratch — even if only one file changed.

✓ The Solution

Add two lines to build-push-action to save layers to GitHub's cache and pull them back on the next run:

```
cache-from: type=gha
cache-to: type=gha,mode=max
```



TTS Research Technology

How Layers Work

- Each Dockerfile instruction (FROM, RUN, COPY) creates one layer
- Docker reuses cached layers until the first instruction that changed
- Everything after the changed instruction gets rebuilt

Where Are Layers Stored?

- **type=gha**: GitHub's own cache servers (10 GB, 7-day expiry) , easiest, no setup
- **type=registry**: Docker Hub (persists longer, uses registry storage)
- **type=local**: runner disk (self-hosted runners only)



Docker images are built in layers. Each Dockerfile instruction creates a layer. If nothing changes in an earlier layer, Docker can reuse that layer instead of rebuilding it. The challenge with GitHub-hosted runners is that every run starts on a fresh virtual machine, so the local cache is gone. To solve this, we can use the GitHub Actions cache with `cache-from: type=gha` and `cache-to: type=gha,mode=max`. This can make repeated builds much faster.

CPU architecture matters

- When using a container, you must choose an image built for the correct CPU architecture of your system, such as **x86_64/amd64** or **arm64**.
- A container does not ignore hardware differences. If the image was built for the wrong architecture, it may fail to start or run much more slowly through emulation.
- For example, a container built for **amd64** may not run natively on an **Apple Silicon Mac**, which uses **arm64**.



TTS Research Technology



CPU architecture matters when building and running containers. An image built for amd64 is not the same as an image built for arm64. Most HPC systems, including Tufts HPC, typically run amd64 images. Apple Silicon Macs use arm64. If users pull an image built for the wrong architecture, it may fail to run or may run slowly through emulation. So when we publish images, we need to be clear about which architectures are supported.

Building Multi-Architecture Images

One image manifest, two architectures: users on Apple Silicon, AWS Graviton, and x86 servers all pull the right variant automatically.

Why It Matters

- Apple Silicon Macs run ARM natively, an amd64-only image runs slowly under emulation
- Tufts HPC and PC can only run amd64 images.
- Docker Hub serves the correct variant automatically via a manifest list

Add to Your Workflow

Step 1 — add before setup-buildx
- uses: docker/setup-qemu-action@v4

Step 2 — add to build-push-action
platforms: linux/amd64,linux/arm64

How It Works

- **QEMU** emulates ARM64 on the x86 runner so both architectures can be built in one job
- **Buildx** builds both variants and pushes them as a single multi-arch manifest to Docker Hub
- **docker pull** automatically fetches the right variant for the host platform

Trade-offs

- ARM build under QEMU emulation is 4–8× slower than native, a 5 min build may take 30–40 min
- For faster ARM builds, use a native ARM self-hosted runner (e.g. Apple M-series or AWS Graviton) with runs-on: [self-hosted, arm64]



TTS Research Technology



Multi-architecture images allow one image name to support multiple CPU architectures. For example, the same Docker Hub image tag can contain both linux/amd64 and linux/arm64 variants. Docker will automatically pull the correct one for the user's machine. This is helpful if users develop on Apple Silicon Macs but deploy to x86 servers. The trade-off is that building ARM images through QEMU emulation can be much slower. For production HPC images, we should always make sure the amd64 image is built and tested.

Key Takeaways

- Containers make research software more portable and reproducible
- GitHub repositories document how containers are built through Dockerfiles and workflow history
- GitHub Actions turns manual build, tag, and push steps into a repeatable automated workflow
- Versioned Docker images make updates easier to track, test, and roll back



TTS Research Technology



04 Demo

<https://github.com/TuftsHPCbioinfo/GitHubActionDemo>

```
cd Downloads
git clone https://github.com/TuftsHPCbioinfo/GitHubActionDemo
docker run --rm \
  -p 8787:8787 \
  -e DISABLE_AUTH=true \
  -v ~/Downloads/GitHubActionDemo:/home/rstudio/workshop_demo \
  tuftsttsrt/rstudio:4.6.0
```



TTS Research Technology



For the demo, I will use this GitHub repository: [TuftsHPCbioinfo/GitHubActionDemo](https://github.com/TuftsHPCbioinfo/GitHubActionDemo). I will clone the repository, run the provided Docker image, and show how the workflow is connected to the repository. The example runs an RStudio container and mounts the demo repository into the container. This gives us a simple way to demonstrate how a containerized environment can be built, published, and used.

Questions?



TTS Research Technology



That concludes the presentation. The main takeaway is that GitHub Actions can turn container builds from a manual process into a repeatable automated workflow. For research technology and HPC, this helps us deliver software environments more reliably, reduce manual maintenance, and create a foundation for more complete CI/CD workflows in the future. I am happy to take questions.